

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ
ФГБОУ ВО Волгоградский государственный технический университет
Кафедра «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ
АВТОМАТИЗИРОВАННЫХ СИСТЕМ»

Манипулирование изменениями программного кода и их отмена

Методические указания



Волгоград
2020

Рецензент

к. т. н., доц. Н.П. Садовникова

Издается по решению редакционно-издательского совета
Волгоградского государственного технического университета

Основы работы в распределенных системах контроля версий: метод. указания / сост. О. А. Сычев, Г.В. Терехов; ВолгГТУ. – Волгоград, 2020. – 36 с.

В методических указаниях изложены цель, подготовка и порядок выполнения лабораторных работ «Поиск ошибок путем бисекции репозитория» и «Манипулирование изменениями и их отмена» по дисциплине «Распределенный контроль версий кода», приведены основные сведения о базовых командах работы в современных системах распределенного контроля версий Git и Mercurial, рассмотрены способы поиска ошибок в коде, отмены изменений, откладывания, пересылки и копирования коммитов, а также очереди патчей.

Методические указания предназначены для изучения дисциплины «Распределенный контроль версий кода» и могут быть использованы при выполнении лабораторной работ «Поиск ошибок путем бисекции репозитория» и «Манипулирование изменениями и их отмена». Методические указания могут быть использованы студентами, обучающимися по направлению 09.03.04 «Программная инженерия», а также широким кругом лиц, интересующихся программированием.

© Волгоградский государственный
технический университет, 2020

Оглавление

1. Поиск ошибок и их источника.....	4
2. Лабораторная работа №4 «Поиск ошибок путем бисекции репозитория».....	10
2.1. Цель работы и подготовка.....	10
2.2. Выполнение работы.....	10
3. Откат и отмена изменений.....	11
4. Откладывание изменений "на полку".....	16
5. Пересылка пакетов изменений.....	20
6. Копирование коммитов.....	25
7. Лабораторная работа №5 «Манипулирование изменениями и их отмена».....	28
7.1 Цель работы и подготовка.....	28
7.2 Выполнение работы.....	28
8. Очереди патчей	29

1. Поиск ошибок и их источника

Системы контроля версий предоставляют несколько инструментов, призванных помочь вам в отладке ваших программ. Это команды аннотации файла, показывающие для каждой строки последний коммит, который ее изменял (`annotate` или `blame`); а также команда поиска ошибки на отрезке коммитов путем деления его пополам. Их использование может значительно упростить отладку кода, в особенности при наличии большого коллектива разработчиков и длительной истории развития программного проекта.

Аннотация файла

Если вы нашли ошибку в коде программы или место, которое вам кажется ошибкой – часто бывает полезно узнать, когда, кем и по какой причине она была внесена. Очень часто то, что кажется вам ошибкой или неэффективным кодом на самом деле предназначено для коррекции поведения программы в каком-нибудь частном случае, о котором вы и не догадываетесь (после работы над проектом год-другой вам может показаться странным и свой собственный код, посвященный частному случаю, о котором вы давно забыли). Другой ситуацией, когда вам нужно найти коммит, вводящий ту или иную строчку, может быть конфликт между двумя подсистемами в программе, написанными разными разработчиками. Зная автора мешающих вам строк, вы можете обсудить с ним проблему и найти решение, устраивающее обе части программы; внося самостоятельно изменения в чужую подсистему вы можете добавить в программу еще большее количество ошибок.

Аннотация файла — самая удобная возможность в этом разобраться. Она покажет вам, какие коммиты изменяли каждую строку файла в последний раз. Зная коммит, в котором была внесена интересующая вас строка, можно найти автора строки и коммит-сообщение. Если вам вдруг попадается фрагмент кода, назначение которого не совсем понятно, благодаря аннотации можно точно узнать автора этого фрагмента, и обратиться сразу к нему – если вы не поймете этого из коммит-сообщения. Регулярное использование аннотаций приучает программистов уделять внимание написанию хороших коммит-сообщений гораздо больше, чем грозные указы руководителя – потому что именно при чтении аннотации эти сообщения становятся полезными.

В завершение общего обзора следует отметить, что команды `annotate` и `blame` относятся к тому небольшому числу команд, для которых использование консоли неудобно. GUI-версии этих команд, имеющиеся во многих пакетах (TortoiseGit, TortoiseHg, Git Gui) намного удобнее, т.к. позволяют прокручивать большие текстовые файлы и

переключаться к аннотации предыдущей версии файла без выполнения команды `update`. Последняя возможность нередко бывает полезной: представьте, что, найдя нужную строчку в аннотации, вы обнаружили, что последним изменением в ней был коммит, исправлявший стиль кодирования. Очевидно, что вас интересует не он, а предыдущие коммиты.

Система `Git` поддерживает два варианта написания команды: `annotate` и `blame` (последнее слово буквально означает «винить кого-либо» и служит иллюстрацией одного из применений этой команды – поиска, кого ругать за вот эту глупость в коде...); они очень схожи, при этом `blame` считается основной командой, а `annotate` поддерживается в основном для совместимости. Если вы используете команду `git blame` из командой строки для большого файла, то важной опцией является `-L <начало>, <конец>`, позволяющая вам указать диапазон строк файла, которые вы хотите увидеть аннотированными, в противном случае результат, выведенный на консоль, может быть очень длинным (см. рис. 1). Начало диапазона всегда указывается как номер строки, если конец это просто число, то оно трактуется как номер строки; если же перед ним стоит знак `+` или `-` то число трактуется как количество строк после (плюс) или перед (минус) «началом», которые надо показать (см. рис. 2).

```
$ git blame -L 23,33 app/models/book_new_camp.rb
2858fb9c (Евгений 2016-03-29 14:02:08 +0300 23)  def pending?
62e371e6 (Евгений 2016-04-07 12:58:30 +0300 24)      self.status == nil
2858fb9c (Евгений 2016-03-29 14:02:08 +0300 25)  end
2858fb9c (Евгений 2016-03-29 14:02:08 +0300 26)
2858fb9c (Евгений 2016-03-29 14:02:08 +0300 27)  def rejected?
2858fb9c (Евгений 2016-03-29 14:02:08 +0300 28)      self.pending? ? false :
!self.stat
2858fb9c (Евгений 2016-03-29 14:02:08 +0300 29)  end
2858fb9c (Евгений 2016-03-29 14:02:08 +0300 30)
2858fb9c (Евгений 2016-03-29 14:02:08 +0300 31)  def accepted?
2858fb9c (Евгений 2016-03-29 14:02:08 +0300 32)      self.status
2858fb9c (Евгений 2016-03-29 14:02:08 +0300 33)  end
```

Рисунок 1. Пример `git blame` с концом интервала строк, как номером строки

```
$ git blame -L 23,+11 app/models/book_new_camp.rb
2858fb9c (Евгений 2016-03-29 14:02:08 +0300 23)  def pending?
62e371e6 (Евгений 2016-04-07 12:58:30 +0300 24)      self.status == nil
2858fb9c (Евгений 2016-03-29 14:02:08 +0300 25)  end
2858fb9c (Евгений 2016-03-29 14:02:08 +0300 26)
2858fb9c (Евгений 2016-03-29 14:02:08 +0300 27)  def rejected?
2858fb9c (Евгений 2016-03-29 14:02:08 +0300 28)      self.pending? ? false :
!self.stat
2858fb9c (Евгений 2016-03-29 14:02:08 +0300 29)  end
2858fb9c (Евгений 2016-03-29 14:02:08 +0300 30)
2858fb9c (Евгений 2016-03-29 14:02:08 +0300 31)  def accepted?
2858fb9c (Евгений 2016-03-29 14:02:08 +0300 32)      self.status
2858fb9c (Евгений 2016-03-29 14:02:08 +0300 33)  end
```

Рисунок 2. Пример `git blame` с концом интервала строк, как смещением

Опция `-M` позволяет игнорировать переносы и копирования строк внутри файла (считая настоящим автором перенесенной или скопированной строки коммит, который ее изменил – а не коммит, который ее переместил или скопировал), опция `-C` старается учитывать также копирование и переименования файлов.

В Mercurial для просмотра авторов каждой строки файла используется команда `annotate`. В ней нет параметра с диапазоном строк, поэтому выводится весь файл целиком (это можно обойти, используя конвейеры в командной строке), зато есть параметр `-r`, позволяющий увидеть аннотацию файла по состоянию на момент указанной ревизии без выполнения `update` к этой ревизии, что может быть очень полезно в случае, если последние изменения в строке не то, что вам нужно.

Бинарный поиск

Аннотирование файла полезно, когда вы знаете, где у вас ошибка, и есть с чего начинать. Если вы не знаете, что у вас сломалось, и с тех пор, когда код наверняка работал, было сделано много коммитов, вы наверняка обратитесь за помощью к бинарному поиску (дихотомии). Бинарный поиск позволяет быстро найти первый коммит, в котором была внесена ошибка, избежав перебора всех ревизий начиная с последней – при условии, что у вас есть относительно быстрый способ проверить, присутствует ли ошибка в программе (юнит-тесты особенно полезны, но ручное тестирование тоже возможно). Также желательна возможность скомпилировать (если требуется) и запустить соответствующий участок кода на любом коммите между гарантированно работающим и гарантированно ошибочным состоянием (так что правило – не коммитить не запускающийся код имеет вполне практическое значение); однако вы можете пропустить ревизии, которые невозможно проверить с помощью опции `skip`. Принцип работы бинарного поиска прост: разделить имеющийся отрезок коммитов пополам и проверить средний коммит – есть в нем ошибка, или нет? Если уже есть, то надо искать источник в первой половине коммитов, если еще нет – во второй, и так далее. Таким способом можно найти нужный коммит за относительно небольшое (логарифмическое относительно количества коммитов в отрезке) количество проверок.

Команда `git bisect` выполняет бинарный поиск по истории коммитов, и призвана помочь как можно быстрее определить, в каком коммите была внесена ошибка.

Положим, вы наткнулись на некорректное поведение вашей программы, и вы не представляете, почему она себя так ведет, но вам удастся воспроизвести ошибку. Можно использовать `bisect`, чтобы

выяснить это (см. рис. 3). Сначала выполните `git bisect start`, чтобы запустить процесс, а затем `git bisect bad`, чтобы сказать системе, что текущий коммит содержит ошибку. Затем необходимо сказать команде `bisect`, когда было последнее известное хорошее состояние при помощи команды `git bisect good`.

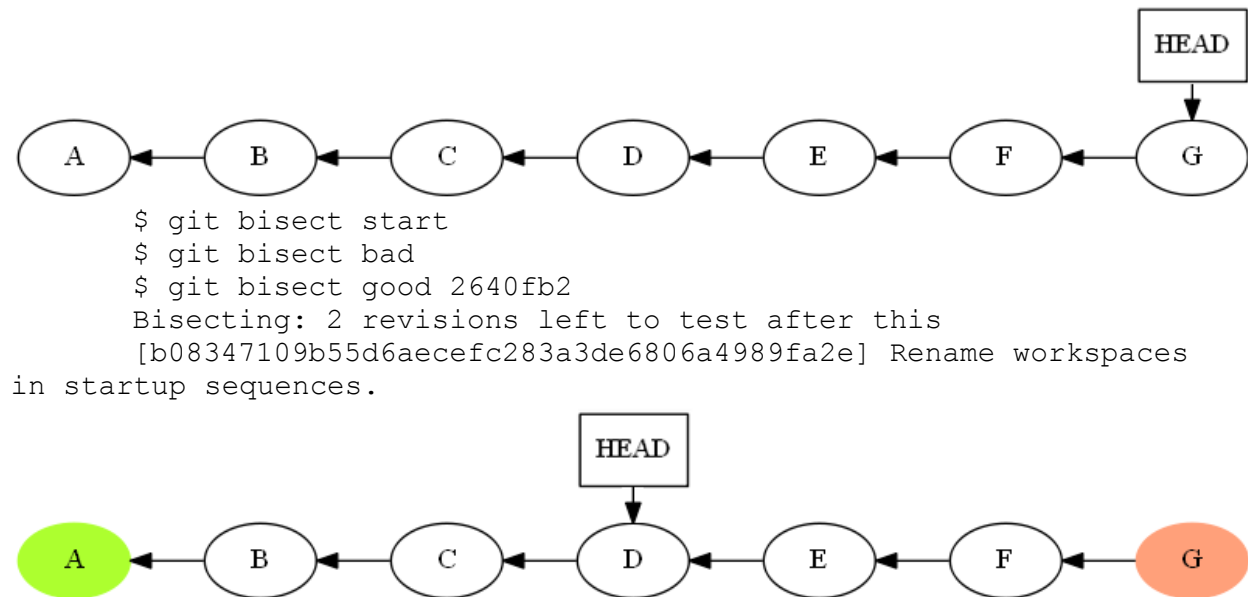


Рисунок 3. Пример `git bisect`

Git определил, что между коммитами, которые были указаны как текущий плохой и последний хороший, было сделано примерно 4 коммита и он обновил код к версии из середины отрезка. Теперь можно запустить свои тесты и/или протестировать код вручную и выяснить, проявляется ли ошибка в этом коммите. Если ошибка проявляется, то она была внесена в проект где-то раньше автоматически определенного коммита. Проверив программу, мы обнаружили, что ошибки в текущем коммите нет. Сообщим Git'у об этом при помощи команды `git bisect good` (см. рис. 4).

```

$ git bisect good
Bisecting: 0 revisions left to test after this
[da0acb20800999558b6302b2dea3e954a9c0ea26] contrib: add per-
workspace-layout.pl
  
```

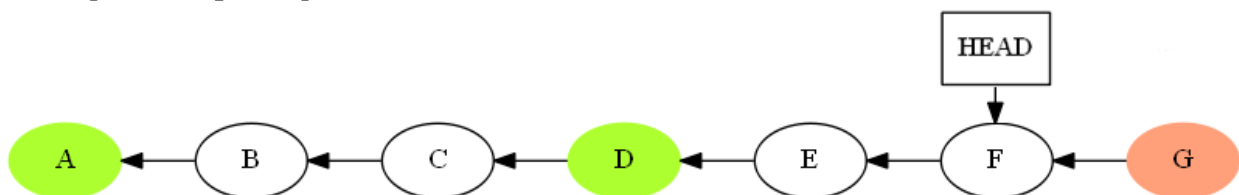


Рисунок 4. Пример `git bisect good`

Теперь вы на коммите посередине между тем, который только что смотрели и тем, который был помечен как плохой. После еще одного

тестирования программы выясняется, что к этому моменту ошибка уже есть. Об этом нужно сообщить Git при помощи команды `git bisect bad` (см. рис. 5).

```
$ git bisect bad
Bisecting: 0 revisions left to test after this
[8d38529bcdafd331a618552399a04a30b71a57c3] tests: move
set_wm_class to X11::XCB::Window
```

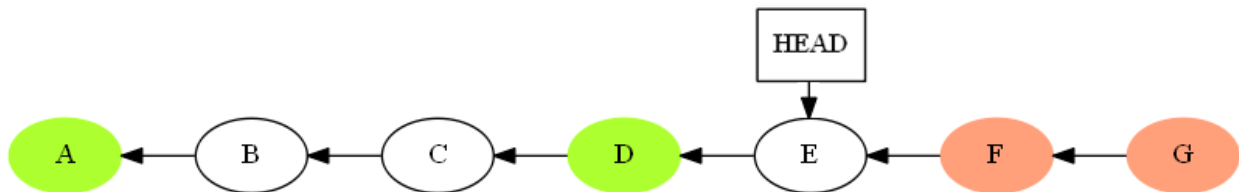


Рисунок 5. Пример `git bisect bad`

Здесь ошибка тоже проявляется; теперь у Git достаточно информации чтобы определить коммит, в котором была произведена ошибка – в данном случае это коммит E, первый из содержащих ошибку. Он выдает нам некоторую информацию о коммите, в том числе и список файлов, которые были изменены (см. рис. 6).

```
$ git bisect bad
a99fc537fc4bd8a0eb3965a2ede342b7806c851e is first bad commit
commit a99fc537fc4bd8a0eb3965a2ede342b7806c851e
Author: Nikolay <nikolay@example.com>
Date: Tue Jan 18 7:27:33 2016 -0800

re-shuffle struct members to save a bit of memory

:040000 040000 493cba4b1f84c0e907a37f5f4e40d3576bb84785
7763c1bb6f7e4e7b1296863b5df945f5b1760f67 M modules
```

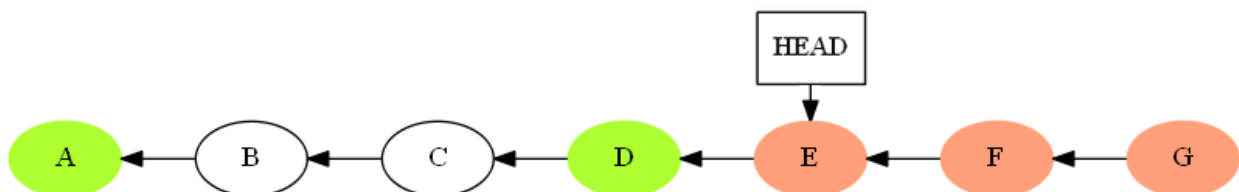


Рисунок 6. Пример вывода информации об автоматически найденном коммите с ошибкой

Иногда в процессе разработки встречаются ревизии, для которых невозможно определить, была ли ошибка: например, если программа не компилируется из-за синтаксических ошибок или в данном состоянии программы временно не работает модуль, в котором находится ошибка. Такие коммиты можно пропустить, используя команду `git bisect skip`. Однако во многих проектах подобных коммитов стараются избегать, в частности принято гарантировать возможность скомпилировать и запустить программу перед каждым коммитом. Команда `skip` использует

псевдослучайные числа, поэтому ее использование снижает эффективность бисекции.

Если вы закончили и разобрались в источнике ошибки, чтобы закончить бисекцию и перейти к исправлению ошибки следует выполнить `git bisect reset`, чтобы переключиться на ту ревизию, где вы были до начала бинарного поиска (см. рис. 7).

```
$ git bisect reset  
Already on 'master'
```

Рисунок 7. Пример `git bisect`

В Mercurial команда `bisect` работает аналогично. При наличии автоматических тестов процесс можно далее автоматизировать, используя скрипты, возвращающие «хорошая» или «плохая» ревизия с помощью числа, возвращаемого при завершении процесса (0 – «хорошая», 125 – «пропустить», 127 – «прервать», остальные числа от 0 до 127 – «плохая»).

2. Лабораторная работа №4 «Поиск ошибок путем бисекции репозитория»

2.1. Цель работы и подготовка

Целью лабораторной работы является практическое освоение студентами команды `bisect` в условиях сложного репозитория.

Для выполнения работы необходим репозиторий Git или Mercurial с актуальным кодом программного проекта, разрабатывавшегося несколькими участниками, с применением веток - а также среды разработки и другое программное обеспечение (например СУБД) для компиляции и запуска проекта из репозитория. Рекомендуется использование программных проектов, разработанных ранее в ходе междисциплинарного курсового проекта или иной формы групповой проектной деятельности.

При подготовке рекомендуется потренироваться в сборке и запуске программы на любой ревизии репозитория, а также изучить работу команды `bisect`.

2.2. Выполнение работы

К началу работы следует показать преподавателю программные проект в его результирующем виде.

На лабораторной работе, преподаватель выдает задание, как правило заключающееся в том, чтобы с помощью команды `bisect` найти первый коммит в котором появилась определенная функция или особенность работы программы. Студент с помощью команды `bisect` ищет соответствующий коммит. Следует обратить внимание на то, что поскольку ищет не ошибка («плохой коммит») а полезная функция («хороший коммит») то смысл команд `good` и `bad` меняется на противоположный.

При оценке выполнения работы учитывается эффективность бисекции и правильность найденного коммита. Ошибки в выполнении бисекции и повторное ее начало снижают оценку.

3. Откат и отмена изменений

Одна из ценных возможностей системы контроля версий – возможность отменить что-то конкретное. Начиная от неудачно сделанного в спешке последнего добавления до серьезного изменения, сделанного несколько лет назад и оказавшегося вредным. Единственное важное требование при отмене – отменяемые изменения должны представлять один коммит или группу последовательных коммитов. Традиционно изменения делятся на изменение или отмену последних коммитов и изменения в глубине дерева.

Один из самых частых случаев отмены происходит тогда, когда вы делаете коммит слишком рано, забыв добавить какие-то файлы, или неправильно написали коммит-сообщение. В системе контроля версий Git если вам нужно поменять последний коммит, вы можете выполнить `commit` с опцией `-amend` (см. рис. 8).

```
$ git commit --amend
```

Рисунок 8. Пример изменения последнего коммита

Эта команда берёт текущее состояние индекса и использует его для модификации последнего коммита. Если после последнего коммита не было никаких изменений в индексе, то вы просто отредактируете коммит-сообщение. Если же вы хотите добавить какие-либо изменения в последний коммит, то вы должны выполнить `git add` до выполнения `git commit --amend`.

Команда `git reset` предназначена для более сложных случаев и имеет три основных опции (`soft`, `mixed` и `hard`) и два дополнительных варианта (`merge` и `keep`). Эта команда позволяет сдвинуть указатель текущей ветки (и HEAD) назад по дереву коммитов, при этом в Git коммиты, находящиеся впереди всех указателей веток считаются недостижимыми и удаляются сборщиком мусора Git. При применении с опцией `--soft` команда `reset` только передвигает указатель, оставляя и файлы в рабочей копии и индекс неизменными. Обычно это используется для более тщательной переделки последнего коммита: вы делаете коммит, потом добавляете изменения (которые должны были быть сделаны в нем изначально, но вы забыли – или заметили ошибку), отменяете коммит с помощью `git reset --soft HEAD^` и производите его заново (см. рис. 9).

```
$ git reset --soft HEAD^
$ git commit -m "Возможность редактирование профиля
пользователя"
[master 7836d9b] Возможность редактирование профиля
пользователя
3 files changed, 52 insertions(+), 1 deletion(-)
create mode 100644 assets/controllers/user_controller.rb
```

Рисунок 9. Пример отмены коммита

Используя откат на несколько коммитов назад с сохранением рабочего каталога и индекса, вы также можете «сжать» несколько последних коммитов в один (см. рис. 10).

```
$ git reset --soft HEAD~3
$ git commit -m "Возможность редактирование профиля
пользователя"
[master 7836d9b] Возможность редактирование профиля
пользователя
3 files changed, 52 insertions(+), 1 deletion(-)
create mode 100644 assets/controllers/user_controller.rb
```

Рисунок 10. «Сжатие» нескольких последних коммитов в один

Опция `--mixed`, применяемая по умолчанию, сбрасывает не только коммит, но и индекс. Самое частое ее применение происходит без отката, когда вам нужно сбросить содержимое индекса (например, вы добавили в индекс файл, который не должен был быть частью этого коммита), выполняется она просто `git reset`.

Команда `git reset --hard` сбрасывает не только историю коммитов и индекс, но и файлы в рабочем каталоге. Она применяется тогда, когда вы реально решили избавиться от последних коммитов, стерев эти изменения отовсюду. С этой командой следует быть осторожным – потерянные изменения вернуть может быть проблематично, если и вовсе невозможно. Однако если вы случайно испортили что-либо в репозитории (неудачным слиянием, например — либо плохо проверив результат слияния при коммите, либо случайно использовав `git pull` вместо `git fetch`), `git reset --hard` – это ваше спасение, позволяющее привести репозиторий в нормальное состояние (см. рис. 11).

```
$ git reset --hard HEAD^
HEAD is now at 47b943b Правка рекламного блока на странице
фильтра перед h1
```

Рисунок 11. Отмена коммитов

Однако при всех своих достоинствах `git reset` имеет два крупных недостатка. Во-первых, вы можете использовать его только локально – т.е. на изменениях, которые не были вытолкнуты (если быть точнее, на изменениях, которые не были никем вытянуты) – в противном

случае вы конечно можете выполнить `push` с опцией `force`, но вот люди, которые уже вытянули отменяемые вами изменения и сделали коммиты поверх них получают проблемы при слиянии ваших изменений со своими. Во-вторых, отменяемые изменения не обязательно находятся в последних коммитах – время от времени вы обнаруживаете, что надо отменить что-либо раньше. Для этого существует команда `git revert` (см. рис. 12). Вместо удаления неправильного коммита команда `revert` пытается его обратить, добавит противоположный коммит – т.е. коммит, который добавляет все удаленные строки и удаляет все добавленные. В коммит-сообщении помимо самого факта реверсии обычно указываются его причины. Если строки, изменения которых вы пытаетесь удалить, изменялись после отменяемого коммита – будьте готовы разрешать конфликты в ходе выполнения `git revert`. Для `revert` так же, как и для `reset` можно указать количество отменяемых коммитов.

```
$ git revert 47b943b169d0d8d9c2a5afd8749ad3ee63b1500e
[master 9d397b6] Revert "Правка рекламного блока на странице
фильтра перед h1"
 2 files changed, 2 insertions(+), 2 deletions(-)
```

Рисунок 12. Отмена изменений при помощи `git revert`

Опция `-m` позволяет вам обращаться коммиты слияния, указывая, какую из двух сливаемых веток следует оставить, а какую – обратить (см. рис. 13). Обращение слияния полностью отменяет слитые при нем изменения (даже если в дальнейшем вы сольете эту ветку снова). Команда `revert` не настолько опасна, как `reset`, потому что все изменения (и их отмены) остаются в истории и, следовательно, сами могут быть отменены, если были сделаны ошибочно.

```
$ git revert -m 1 47b943b169d0d8d9c2a5afd8749ad3ee63b1500e
Finished one revert.
[master 88edd6d] Revert "Merge branch 'extranet'"
1 files changed, 0 insertions(+), 2 deletions(-)
```

Рисунок 13. Пример обращения коммитов `git revert` с опцией `-m`

Пример репозитория после отмены слияния представлен на рисунке 14. Коммит «Е» является результатом слияния ветки `master` и `extranet`. После слияния в `master` было сделано несколько коммитов, после чего возникла потребность отменить слияние с веткой `extranet`, результирующий коммит «Е'» содержит отмену коммитов «G» и «F». Обратите внимание, что это эквивалентно удалению этого кода программистом, а не удалению слияния, так что вернуть этот код новым слиянием ветки `extranet` нельзя; для возврата этого кода необходимо удалить или обратить коммит «Е'».

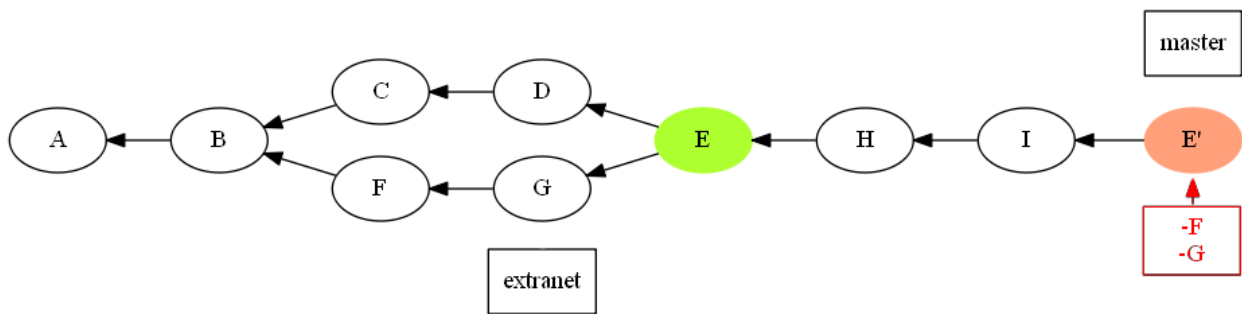


Рисунок 14. Пример репозитория после отмены мержа

Mercurial строже, чем Git подходит к сохранности истории, однако он предоставляет набор инструментов отмены изменений. Команда `hg rollback` позволяет отменить последнюю транзакцию – команду, изменившую ваш репозиторий. Чаще всего это либо `commit` (после чего вы можете переделать последний коммит), либо `pull` (очень полезно, если вы случайно затянули себе в репозиторий что-то не то). Состояние рабочего каталога остается прежним, так что изменения в файлах вы не потеряете (но если вы делали `pull --update` (см. рис. 15), то затянутые изменения останутся в рабочем каталоге и их надо изменять отдельно). Важно, чтобы перед выполнением `rollback` вы не вносили дополнительных изменений в репозиторий – два раза подряд эта команда не работает. В частности, это относится к ситуации, когда вы хотите отменить `pull`, но уже сделали `merge` и `commit` этого слияния – отменится лишь коммит, но не затягивание. Поскольку она реально удаляет изменения, то лучше не выполнять ее на вытолкнутых изменениях.

```

$ hg pull --update repol.2
pulling from repol.2
searching for changes
adding changesets
adding manifests
adding file changes
added 1 changesets with 1 changes to 1 files
1 files updated, 0 files merged, 0 files removed, 0 files
unresolved
$ hg rollback
repository tip rolled back to revision 1 (undo pull)
working directory now based on revision 1

```

Рисунок 15. Пример отмены последнего затягивания в репозитории

Команда `hg revert` в некотором смысле противоположна `rollback` – она не трогает историю коммитов, а сбрасывает изменения в файлах рабочей копии – то есть возвращает их в состояние последнего коммита. Это бывает полезным, если вы начали что-то делать, после чего запутались и решили вернуться к стабильной версии (см. рис. 16). Не

путайте эту команду с `git revert`, они совершенно разные – эквивалентом `hg revert` будет `git reset --hard HEAD`.

```
$ hg revert -a
```

Рисунок 16. Пример сброса изменения в файлах рабочей копии на один коммит

Для сброса содержимого файлов к состоянию в конкретной ревизии нужно использовать ключ `-r` и указать номер ревизии (см. рис. 17).

```
$ hg revert -r 9e7c1d1718c1
```

Рисунок 17. Пример сброса изменения в файлах рабочей копии до конкретной ревизии

Аналогом `git revert` является команда `hg backout` (см. рис. 18, 19), создающая противоположные изменения. Поведение ее немного отлично от `git revert` в случае, если удаляемый коммит не последний. Она делает «обращающий» коммит не потомком последнего коммита в ветке, а потомком удаляемого коммита, после чего эти изменения сливаются, используя `merge` с теми, которые стояли позже (вручную, либо используя `hg backout -merge` (см. рис. 21)); в любом случае изменения происходят только в файлах рабочего каталога (чтобы вы могли их проверить) и их нужно закоммитить. Таким образом `hg backout` увеличивает количество шагов, но дает вам возможность протестировать работу программы после отмены изменений. При отмене слияния эффект тот же, что и в `git revert`: отмененные изменения теряются навсегда, повторное слияние ветки их не вернет.

```
$ hg backout -r "back out" tip
reverting .hgtags
changeset      6073:1f70d9327aa3      backs      out      changeset
6072:b6ba3bc09256
```

Рисунок 18. Отмена последнего изменения

```
$ hg backout -r b6ba3bc09256
reverting .hgtags
1 files updated, 0 files merged, 0 files removed, 0 files
unresolved
```

Рисунок 19. Отмена конкретного изменения

Чтобы отменить изменение сразу применив слияние, вызовем команду `hg backout -merge`. Состояние репозитория до вызова `hg backout` представлено на рисунке 20.

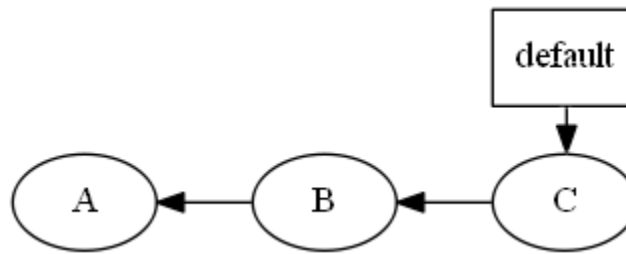


Рисунок 20. Состояние репозитория до отмены изменений

Допустим мы хотим отменить коммит «В». Применим команду `hg backout -merge`, как показано на рисунке 21.

```

$ hg backout --merge -m 'second back out' b6ba3bc09256
reverting user_controller.rb
created new head
changeset 3:5d235a7062b8 backs out changeset 6072:b6ba3bc09256
merging with changeset 3:5d235a7062b8
merging user_controller.rb
0 files updated, 1 files merged, 0 files removed, 0 files
unresolved
(branch merge, don't forget to commit)
  
```

Рисунок 21. Отмена последнего изменения с выполнением слияния

В результате получим репозиторий с коммитом слияния, вид которого представлен на рисунке 22. Коммит «Е» - коммит, образованный в результате отмены изменений, коммит «F» - результат слияния коммита «Е» и «D», как последнего коммита в репозитории.

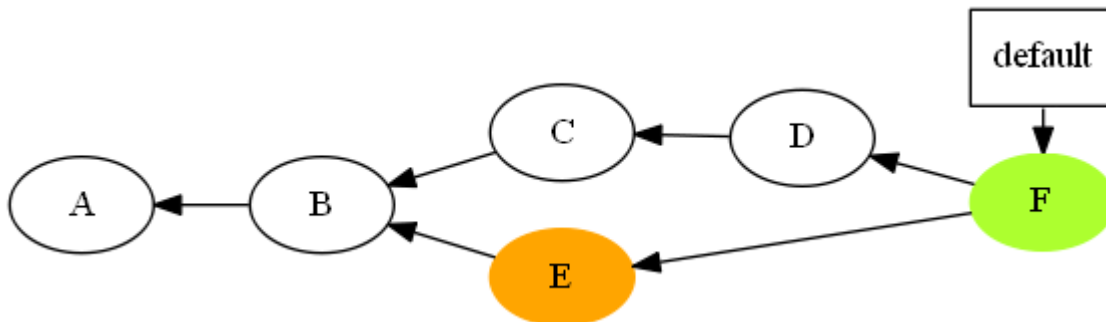


Рисунок 22. Состояние репозитория после отмены изменений с мержем

4. Откладывание изменений "на полку"

При работе над большим программным проектом, очередное изменение может затрагивать большое количество файлов и занимать много времени. Периодически возникают случаи, когда прямо посреди работы необходимо прерваться и заняться чем-то другим, после чего вернуться и закончить исходную работу. Это может быть срочное сообщение о критически важной ошибке или необходимость вытянуть и проверить код от ваших коллег. Делать частичный коммит (с возможным слиянием с последующим в дальнейшем) не лучший вариант. Вместо

этого, недоделанные изменения можно «спрятать» или «отложить на полку».

Еще одна ситуация с временным откладыванием изменений получается, когда изменения были произведены на неправильной ветке, причем выяснилось это уже после внесения изменений в код (вполне возможно, что конкретно изменяемые участки кода одинаковы в обеих ветках). Чтобы избежать их стирания при переключении на нужную ветку, вы должны сначала «спрятать» изменения, потом переключить на ветку, после чего вернуть «спрятанные» изменения.

В системе Git для этого используется команда `stash`. Предположим, что работаете над двумя файлами в проекте и, возможно, добавляете в индекс одно из изменений. Текущее состояние покажет `git status` (см. рис. 23).

```
$ git status
On branch master
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    modified:   app/models/camp.rb

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)

    modified:   app/models/ability.rb
```

Рисунок 23. Пример применения команды `git status`

Теперь вам нужно заняться другой работой, но коммит вы делать не хотите. Чтобы спрятать все сделанные изменения, выполните `git stash`. При этом в специальную «закладку» помещаются изменения файлов в рабочем каталоге и состояние индекса.

В данный момент вы легко можете переключать ветки и работать над другими проблемами; ваши изменения сохранены в специальном стеке. Чтобы посмотреть, что у вас есть припрятанного, используется `git stash list` (см. рис. 24).

```
$ git stash list
stash@{0}: WIP on master: 6ea9b5f Исправлен внешний вид трансфера
stash@{1}: WIP on master: 1f0827e Добавлено поле трансфер
stash@{2}: WIP on master: 68b8e64 Новые блоки с рекламой
```

Рисунок 24. Пример применения команды `git stash list`

В нашем случае два набора изменений были спрятаны ранее, так что у вас теперь три разных припрятанных работы. При возвращении изменений назад у вас есть выбор: если вы хотите вернуть спрятанные изменения и убрать их со стека, то используется команда `git stash`

pop; если же вы хотите, чтобы изменения остались на стеке – `git stash apply`. Обеим командам необходимо указать опцию `--index` если вы хотите восстановить не только содержимое файлов, но и состояние индекса. Если вы хотите восстановить из стека не последнюю работу, то следует прямо указать ее имя – например `git stash apply stash@{2}`. Если файлы в рабочем каталоге отличаются от состояния последнего коммита перед припрятыванием, то при выполнении команд `pop` или `apply` могут возникать конфликты слияния, которые надо разрешить перед коммитом.

Для того, чтобы просто удалить спрятанную работу используется команда `git stash drop` (при необходимости указывается ее имя).

Команда `git stash branch <имя ветки>`, аналогична `git stash pop`, но создаст вам новую ветку с началом из того коммита, на котором вы находились в то время, когда отложили работу, восстановит в ней вашу работу и затем удалит спрятанное, если оно применилось успешно (см. рис. 25).

```
$ git stash branch new_extranet
Switched to a new branch "new_extranet"
On branch new_extranet
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    modified:   app/models/camp.rb

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in
working directory)

    modified:   app/models/ability.rb

Dropped refs/stash@{0}
(f3af1ebe057dce37efcee700b67b8c2d18f58ff8)
```

Рисунок 25. Пример применения команды `git stash branch`

В Mercurial имеется своя аналогичная команда: здесь вместо припрятывания мы откладываем изменения на полку - `hg shelve`. Эта команда является отдельным расширением, которое надо включить стандартным способом.

Чтобы отложить изменения на полку, нужно вызвать `hg shelve`. Чтобы посмотреть список отложенных изменений: `hg shelve -list`. Чтобы применить отложенные изменения: `hg unshelve`.

Можно откладывать различные изменения и давать им имена. В этом случае команды будут выглядеть как показано на рисунке 26.

```
hg shelve --name <name>
hg unshelve <name>
```

Рисунок 26. Команды shelve и unshelve

Поскольку в Mercurial, в отличие от Git, нет индекса, то команда shelve может использоваться если вам нужно разбить текущие изменения на два коммита: откладываем одну часть изменений на полку, коммитим вторую, потом забираем отложенные изменения и коммитим их. Такая ситуация очень часто возникает при необходимости делать атомарные коммиты, когда программист совершил несколько изменений прежде, чем делать коммит.

Допустим наши изменения в файле привели к возникновению трех ханков, но мы хотим закоммитить первые два одним коммитом, а третий - другим. Для этого выполним команду `hg shelve --append <name>`, указав в качестве параметра `<name>` имя файла, для которого хотим отложить на полку ханки (см. рис. 27).

```
$ hg shelve --append mycode.cpp
diff --git a/mycode.cpp b/mecode.cpp
3 hunks, 6 lines changed
shelve changes to mycode.cpp'? [Ynsfdaq?] y
@@ -5,7 +5,7 @@
{
    float a,b,c,d;

-    a=b=c=d=-1;
+    a=b=c=d=0;

    input_printf("Type 4 float numbers (from 0 to 25). a,b,c,d:\n");
    scanf("%e",&a);
shelve this change to 'Lab2_Den166.cpp'? [Ynsfdaq?] n
@@ -18,7 +18,7 @@
    c<=0 || c > 25 ||
    d<=0 || d > 25 )
{
-    error_printf("wrong input data\n");
+    error_printf("invalid input data\n");
}
else
{
shelve this change to 'Lab2_Den166.cpp'? [Ynsfdaq?] n
@@ -34,6 +34,6 @@
    input_printf("\n");
}

-    return 0;
+    return 1;
}

shelve this change to 'mycode.cpp'? [Ynsfdaq?] y
```

Рисунок 27. Пример применения команды `hg shelve -append`

Сначала нужно указать, что мы хотим сделать с этим файлом. Доступно 8 опций:

5. `y` – отложить изменения на полку;
6. `n` – пропустить изменения;
7. `s` – пропустить оставшиеся изменения в этом файле;
8. `f` – отложить оставшиеся изменения в этом файле;
9. `d` – закончить, пропустить оставшиеся изменения и оставшиеся файлы;
10. `a` – отложить все изменения во всех оставшихся файлах;
11. `q` – завершить, ничего не откладывать;
12. `?` – посмотреть справку.

Мы будем откладывать изменения, поэтому выбираем опцию `y`. Далее утилита отображает ханк, мы хотим отложить только третий ханк, поэтому пропускаем первые два опцией `n`, для третьего применим опцию `y`. Теперь третий ханк отложен на полку. Можно коммитить.

Закоммитив первые два ханка, перейдем к коммиту третьего. Достанем его из хранилища, выполнив команду `hg unshelve -n <name>` (см. рис. 28). Мы не давали никакого имени для нашего отложенного изменения, по умолчанию оно будет называться `default`.

```
$ hg unshelve -n default
unshelve completed
```

Рисунок 28. Пример применения команды `hg unshelve -n`

Изменения были успешно применены к нашему файлу. Теперь можно сделать отдельный коммит из этого изменения со своим сообщением.

5. Пересылка пакетов изменений

Возможны ситуации, когда необходимо синхронизировать хранилища без серверов или вообще без сетевого подключения, используя перенос файлов или пересылку их по электронной почте. Одним из вариантов является копирование полного репозитория, однако в больших проектах копия репозитория может занимать много места. Для создания частичных архивов (пакетов) с изменениями используется команда `bundle`. Репозиторий, из которого создается пакет, будем рассматривать как апстрим репозиторий; репозиторий к которому применяется пакет – как даунстрим.

При создании пакетов с изменениями следует тщательно отслеживать точку начала, с которой записываются изменения в пакет – его базу. Для корректного применения к другой копии репозитория (даунстрим) база должна содержать все коммиты между последним изменением в даунстрим-репозитории и теми коммитами, которые вы скопировали из апстрима. В противном случае применение пакета будет

невозможно. Если такие передачи между репозиториями осуществляются на регулярной основе, то для слежения за точкой последней синхронизации обычно используется специальный тег.

В системе Git команда `git bundle create` позволяет создать пакет, с указанием диапазона входящих в него коммитов. Предположим последний раз, когда осуществлялся обмен между апстрим и даунстрим репозиториями, они оба имели по два коммита. Для слежения за тем, какие коммиты уже были отправлены, используется тег `bundled`, который сейчас указывает на второй коммит. В дальнейшем апстрим-разработчик добавил еще 3 коммита в апстрим репозиторий и теперь хочет создать пакет (см. рис. 29, 30).

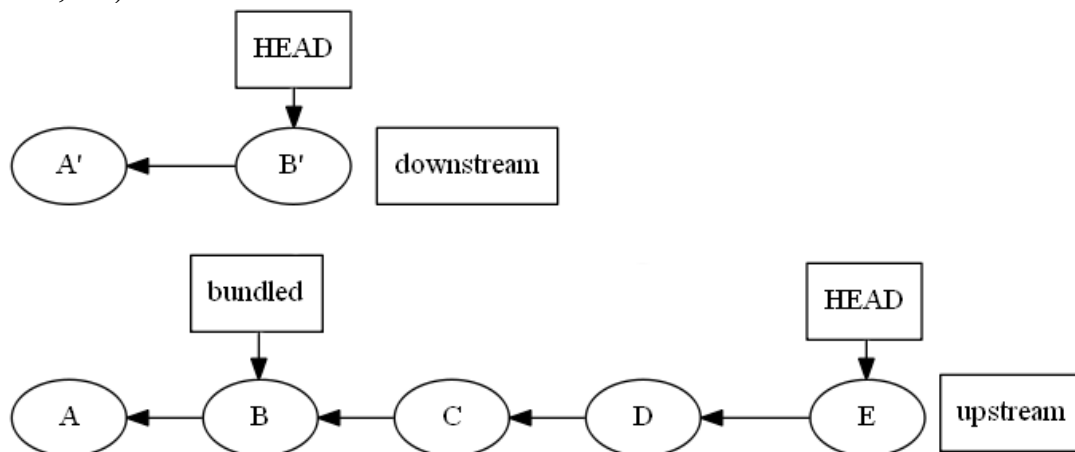


Рисунок 29. Пример двух репозитиев до создания пакета

Наш апстрим репозиторий содержит пять коммитов, представленных на рисунке 30.

```

$ git log --oneline
0272925 add connection check
6691b24 delete deprecated code
7606235 new function
13ec5ec code refactor
79f54f6 initial commit

```

Рисунок 30. Коммиты апстрим репозитория

Допустим, мы хотим отправить коммиты, представленные на рисунке 31.

```

$ git log --oneline bundled..master
0272925 add connection check
6691b24 delete deprecated code
7606235 new function

```

Рисунок 31. Список коммитов апстрим репозитория для отправки
Упакуем наши новые коммиты, как показано на рисунке 32.

```

$ git bundle create newcommits.bundle bundled..master
Counting objects: 9, done.

```

```

Compressing objects: 100% (5/5), done.
Writing objects: 100% (7/7), 621 bytes, done.
Total 7 (delta 3), reused 0 (delta 0)

```

Рисунок 32. Создание пакета в Git

Перенесем тег `bundled` апстрим репозитория чтобы показать, что новые коммиты были упакованы (см. рис. 33).

```
$ git tag --force bundled 0272925
```

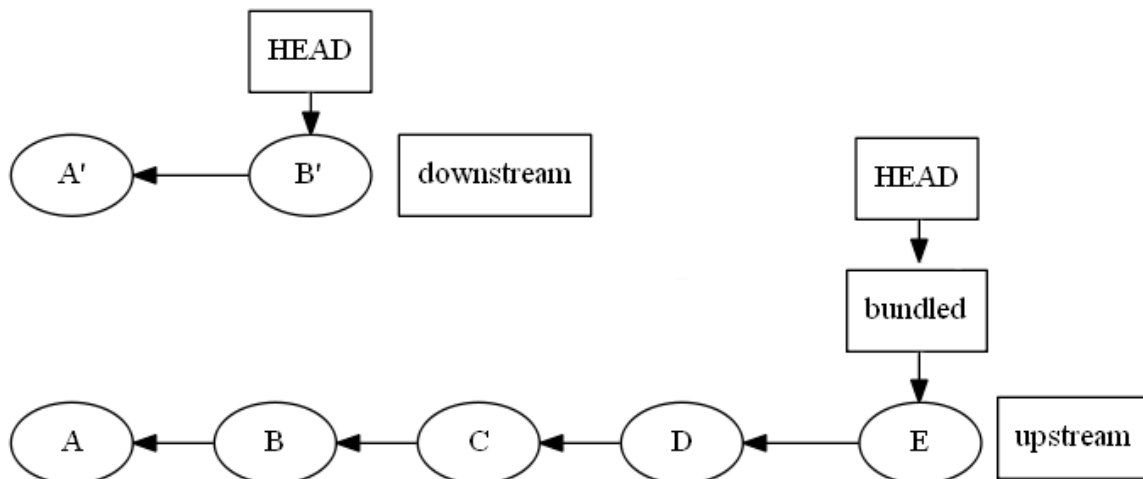


Рисунок 33. Пример двух репозитиев после создания пакета

Рассмотрим другой пример создания пакета – упаковка ветки `feature`, созданной разработчиком апстрима. Состояние репозитиев апстрима и даунстрима показано на рисунке 34.

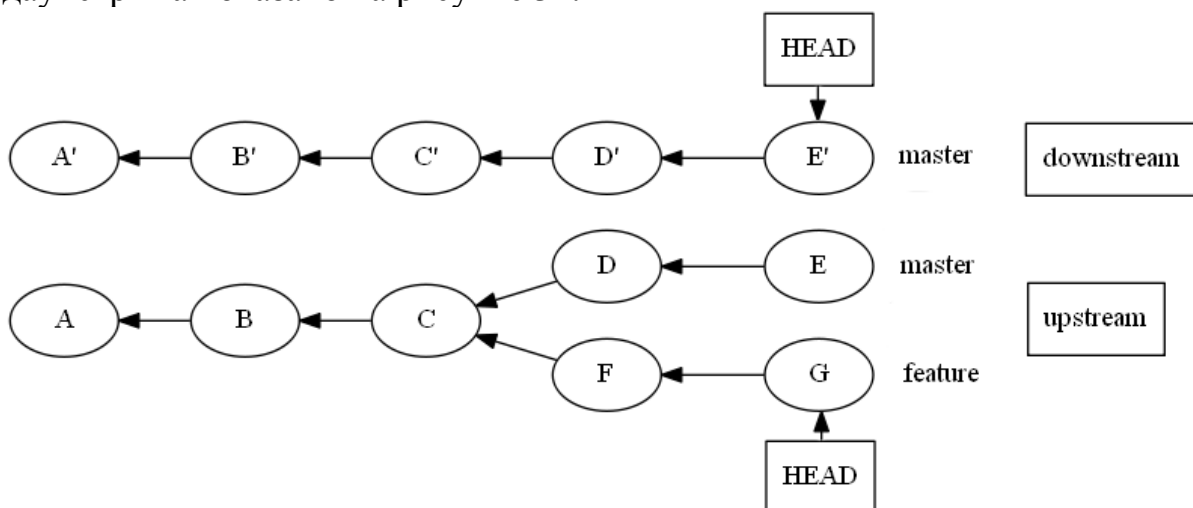


Рисунок 34. Состояние апстрим и даунстрим репозитиев

Упакуем наши коммиты, как показано на рисунке 35.

```

$ git bundle create feature_branch.bundle master..feature
Counting objects: 10, done.
Compressing objects: 100% (5/5), done.

```

```
Writing objects: 100% (7/7), 621 bytes, done.  
Total 7 (delta 3), reused 0 (delta 0)
```

Рисунок 35. Упаковка ветки в Git

После передачи файла пакета, даунстрим-разработчик может использовать `git fetch` или `pull` для получения коммитов в своей репозиторий, используя пакет в качестве удаленного сервера (remote) (см. рис. 36, 37).

```
$ git fetch feature_branch.bundle master  
Receiving objects: 100% (7/7), done.  
Resolving deltas: 100% (3/3), completed with 1 local objects.  
From feature_branch.bundle
```

```
* branch          master      -> FETCH_HEAD
```

Рисунок 36. Пример `git fetch`

```
$ git log --oneline  
13ec5ec code refactor  
79f54f6 initial commit  
git pull feature_branch.bundle master  
From newcommits.bundle  
* branch          master      -> FETCH_HEAD  
Updating 13ec5ec..0272925  
Fast-forward  
 index.php |      1 -  
 1 file changed, 1 deletion(-)  
$ git log --oneline  
0272925 add connection check  
6691b24 delete deprecated code  
7606235 new function  
13ec5ec code refactor  
79f54f6 initial commit
```

Рисунок 37. Пример `git pull`

В системе Mercurial вы также можете создать пакет командой `hg bundle`, для чего следует указать коммит, который вы желаете передать (опция `-r`) и базу (опция `-b`) – коммит, который гарантированно присутствует в даунстрим-репозитории (см. рис. 38). Опция `--all` позволяет создать пакет из всех изменений в репозитории. Создание пакетов для примеров выше будет выглядеть следующим образом.

Наш апстрим репозиторий содержит пять коммитов, показанных на рисунке 38.

```
$ hg log  
changeset: 5:cd8c634f1c80  
tag:      tip  
user:     First Last <email@address.com>  
date:     Sun Feb 19 22:13:02 2017 +0300  
summary:  add connection check
```

```

changeset: 4:342924494a21
user:      First Last <email@address.com>
date:      Sun Feb 19 22:12:41 2017 +0300
summary:   delete deprecated code

changeset: 3:10868ddb4b90
user:      First Last <email@address.com>
date:      Sun Feb 19 22:09:32 2017 +0300
summary:   Added tag bundled for changeset 615f7e5d22e7

changeset: 2:615f7e5d22e7
tag:       bundled
user:      First Last <email@address.com>
date:      Sun Feb 19 22:07:22 2017 +0300
summary:   new function

changeset: 1:41d9fed6b524
user:      First Last <email@address.com>
date:      Sun Feb 19 22:06:41 2017 +0300
summary:   code refactor

changeset: 0:ea7284f23404
user:      First Last <email@address.com>
date:      Sun Feb 19 22:06:04 2017 +0300
summary:   initial commit

```

Рисунок 38. Список коммитов апстрим репозитория

Определимся с коммитами, которые ходим упаковать (см. рис. 39).

```

$ hg log -r ">::branch(default)-::~bundled"
changeset: 3:10868ddb4b90
user:      First Last <email@address.com>
date:      Sun Feb 19 22:09:32 2017 +0300
summary:   Added tag bundled for changeset 615f7e5d22e7

changeset: 2:615f7e5d22e7
tag:       bundled
user:      First Last <email@address.com>
date:      Sun Feb 19 22:07:22 2017 +0300
summary:   new function

changeset: 4:342924494a21
user:      First Last <email@address.com>
date:      Sun Feb 19 22:12:41 2017 +0300
summary:   delete deprecated code

changeset: 5:cd8c634f1c80
tag:       tip
user:      First Last <email@address.com>
date:      Sun Feb 19 22:13:02 2017 +0300
summary:   add connection check

```

Рисунок 39. Список коммитов для упаковки

Упакуем коммиты, как показано на рисунке 40.

```

$ hg bundle -r ">::branch(default)-::~bundled" -a newcommits.hg
6 changesets found

```

Рисунок 40. Пример создания пакета в Mercurial

Применение пакетов осуществляется командой `hg unbundle`, которой нужно указать имя файла пакета (см. рис. 41). Если вы хотите выполнить `update` после `unbundle`, вы можете указать опцию `-u`.

```
$ hg unbundle newcommits.hgadding changesets
adding manifests
adding file changes
added 2 changesets with 2 changes to 2 files
(run 'hg update' to get a working copy)
```

Рисунок 41. Применение пакетов в Mercurial

Кроме создания пакетов, для передачи изменений Mercurial может создавать последовательность патчей используя команду `hg export` (см. рис. 42); впоследствии эти патчи могут быть применены с помощью `hg import` (см. рис. 43), которая, по сути, является синонимом для `patch`.

```
$ hg export -r ">::branch(default)-::bundled" -a -o
newcommits.patch
```

Рисунок 42. Создание последовательности патчей в Mercurial

```
$ hg import newcommits.patch
applying newcommits.patch
```

Рисунок 43. Применение последовательности патчей в Mercurial

6. Копирование коммитов

Существуют множество сценариев, когда вам нужна не вся ветка, а только отдельные коммиты из нее, для перенесения в другую ветку или репозиторий. Одним из частых случаев является перенос исправлений ошибок в ситуации, когда поддерживается несколько версий программы (задействована стратегия `major release – branch`). Если, например, у вас есть две ветки с последними стабильными версиями (5 и 6, например) и `master (default)` с разрабатываемой новой версией (в данном примере она получит номер 7), а в это время пользователи сообщают вам об ошибках в версиях 5 и 6, то часть ошибок потребует одинаковых исправлений во всех трех версиях, часть в двух соседних, и только незначительное количество ошибок нужно будет исправлять лишь в одной ветке, т.к. две других имеют этот код измененным. Было бы неудобно повторять работу по исправлению ошибок в каждой версии. Вместо этого можно выполнить и закоммитить исправление в одной ветке, а потом скопировать этот коммит в остальные.

Другим случаем, когда перенос коммитов необходим, является стратегия `developer-branch (без issue-branch)` – в таком случае периодически будут возникать ситуации, когда ветка вашего подчиненного

содержит плохой код и не может быть слита полностью, но часть коммитов из нее нужна прямо сейчас (исправление срочных ошибок, разблокирование других разработчиков) и эти то коммиты в порядке. Аналогичная ситуация может возникнуть, если при разработке большой feature-branch вы не успели ее завершить к сроку релиза и не может слить в release (production) ветку, однако некоторые сделанные при этом изменения являются полезными и достойны включения в релиз отдельно. Еще одним примером может служить ситуация, когда, разрабатывая приложение с открытым исходным кодом, вы наткнулись на ветку репозитория с чьими-то (возможно, давно брошенными) доработками, некоторые из которых вам кажутся очень полезными – но далеко не все.

Во всех этих случаях полезно использования команды копирования коммитов между ветками `git cherry-pick` и `hg graft`. Смысл их работы состоит в применении `diff` от заданного коммита (набора коммитов) к текущей ветке (при этом может потребоваться разрешать конфликты). Единственный недостаток этих команд проявляет себя в той ситуации, когда вы впоследствии сливаете ветку-источник, из которой копировали коммиты, с веткой-приемником, куда копировали. Поскольку обе ветки содержат две копии одного коммита без всякого указания связи между ними, то при слиянии возникнет множество конфликтов между одинаковыми строками, добавленными в двух разных ветках: с точки зрения системы контроля версий — это независимые действия. Их разрешение является тривиальным – оставить по одной копии каждой строки.

Команда копирования коммитов в Git называется `cherry-pick`. Для своего применения она требует чистого (без незакоммитченных изменений) рабочего каталога и индекса, и нахождения на вершине ветки-приемника копируемого коммита. В самой команде необходимо указать коммит-источник. Если вы хотите изменить коммит-сообщение, то можете воспользоваться опцией `--edit`, опция `-x` автоматически добавит в конец сообщения надпись «cherry picked from ...» с указанием коммита-источника.

Например, пусть мы находимся в узле H (см. рис. 44).

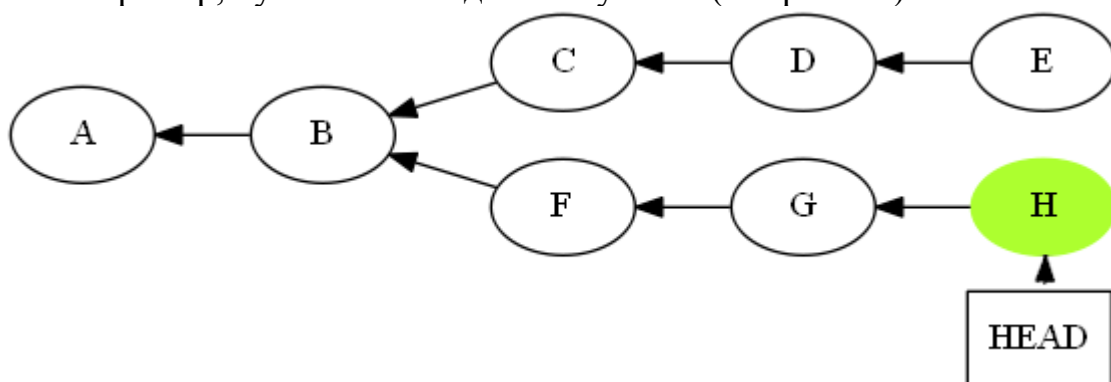


Рисунок 44. Дерево коммитов

Команда `git cherry-pick D`, добавит копию коммита D в текущую ветку (см. рис. 45).

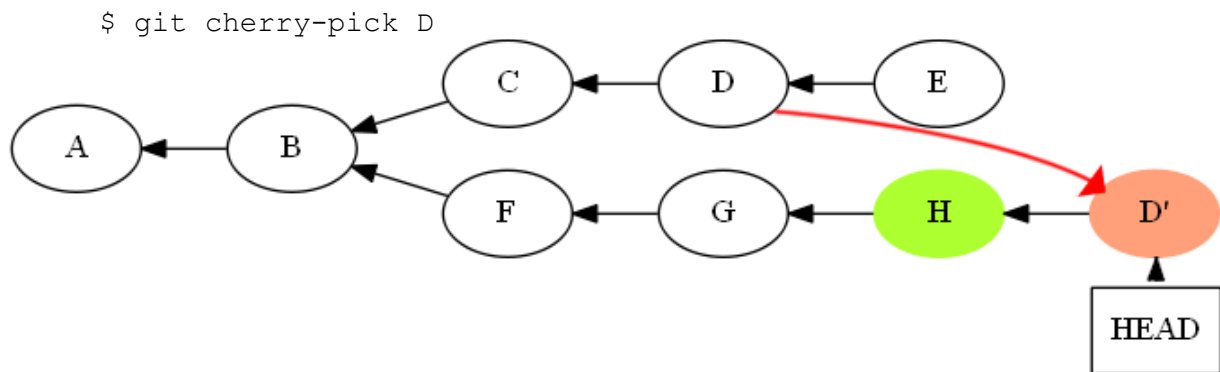


Рисунок 45. Дерево коммитов с копией коммита D

Также `git cherry-pick` позволяет скопировать одновременно несколько коммитов (см. рис. 46).

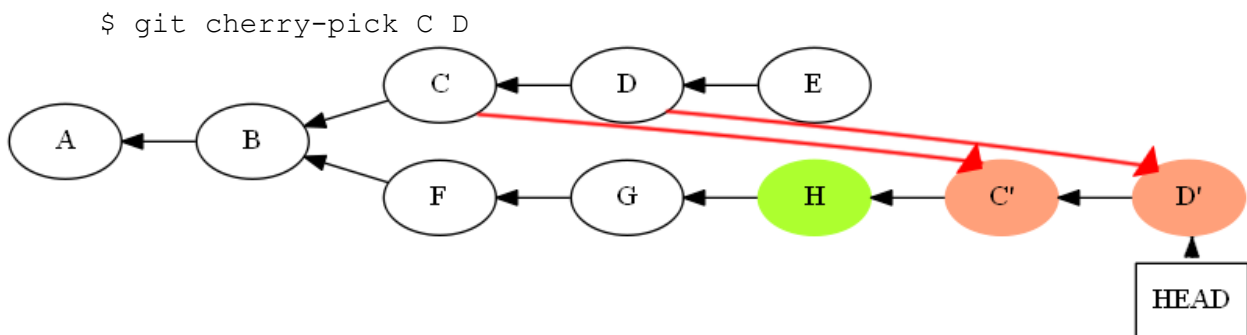


Рисунок 46. Дерево коммитов с копией коммитов C и D

В Mercurial имеется аналогичная команда `hg graft`, которая копирует изменения из одной ветки в другую. Для указания коммита (коммитов) для копирования используется опция `-r`. Опция `--edit` аналогично `git cherry-pick` позволяет отредактировать коммит, опция для добавления записи о копировании в сообщение называется `--log`.

7. Лабораторная работа №5 «Манипулирование изменениями и их отмена»

7.1 Цель работы и подготовка

Целью работы является выработка и закрепления навыков манипуляции изменениями в программном коде в виде коммитов и их отмены без нарушения целостности репозитория.

Для выполнения работы необходим репозиторий Git или Mercurial с несколькими ветками.

При подготовке к работе необходимо научиться выполнять следующие действия:

- 1) откладывание коммитов «на полку» (shelve, unshelve, stash),
- 2) копирование отдельных коммитов с одной ветки на другую (graft, cherry-pick),
- 3) экспорт и импорт коммитов (export, import, bundle)
- 4) отмена изменений путем создания противоположного коммита (backout, revert),
- 5) полное удаление коммитов (rollback, reset, commit --amend).

7.2 Выполнение работы

При выполнении работы преподаватель выдает два задания — одно на манипуляцию изменениями (откладывание с возвращением, копирование, экспорт и импорт), другое на отмену изменений. Оценивается правильность выполнения команд и определения их параметров, а также возможного разрешения конфликтов. Повторный запуск команд приводит к снижению оценки.

Перед выполнением команд рекомендуется сделать копию репозитория на случай неудачного выполнения работы.

8. Очереди патчей

В реальной рабочей практике, нередко приходится работать с кодом программ, находящимся вне вашего контроля. Чаще всего это актуально, если вы внедряете или используете ПО с открытым исходным кодом, либо получили исходный код при покупке ПО по договоренности с разработчиком. Вам приходится делать изменения в коде, которые как устраняют неприятные для вас ошибки, так и добавляют (или изменяют) функциональность системы в нужную для вашей организации сторону. Однако у вас нет контроля над самой внедряемой программой, вы находитесь даунстрим. Какие-то из ваших исправлений и доработок (т.е. по сути патчей) могут быть универсально полезными и быть принятыми в исходный код базовой программы (апстрима) – но процесс этот часто небыстрый; некоторые патчи нужны только вам и в состав апстрима быть предложены просто не могут.

Зато, находясь даунстрим относительно активно развивающегося приложения, вы вынуждены регулярно переносить свои исправления поверх новой версии и тестировать там. Вы можете, конечно, хранить их в виде ветки, которая регулярно перебазируется на вершину апстрима, но, как правило, таких возможностей не хватает. Очень часто при выходе новой версии вам приходится разбираться с каждым патчем в отдельности и решать, стоит ли его сохранять или он более не нужен (или нуждается в переработке). Ситуация осложняется, если вам нужно поддерживать несколько вариантов модификации с различными свойствами (какие-то патчи общие для всех, какие-то специфичные для конкретных систем) – так часто обстоят дела в консультативной фирме, занимающейся внедрением какого-либо ПО для разных предприятий.

Что хотелось бы иметь в такой ситуации, так это набор легко редактируемых (мало ли как их потребуется менять при переходе на новую версию апстрима) патчей, которые можно индивидуально включать и выключать, подготавливая таким образом различные версии программы для тестирования или передачи конкретному заказчику. Первоначально для этих ситуаций была разработана программа (по сути – группа скриптов) quilt, позволявшая управлять последовательно применяемым набором (очередью) патчей, имея возможность произвольно включать и выключать их, однако эта программа не работала с контролем версией. В дальнейшем на ее основе было написано расширение MessageQueue (MQ), которое поддерживало все функции quilt, но было интегрировано с системой контроля версий и позволяло легко преобразовывать патчи в коммиты.

Управление наборами патчей также стало популярно среди программистов, желающих иметь на вершине разработки набор легко изменяемых «коммитов», на которые не распространяется обычный

принцип неизменности истории (что особенно важно для Mercurial). Коммиты в виде патчей легко включаются, выключаются и редактируются, а после окончательной реализации всех необходимых возможностей, в «чистовом» варианте могут быть преобразованы в обычные неизменяемые коммиты.

Все эти преимущества достигаются за счет проблем со слиянием и разрешением конфликтов, которые приходится делать во многом вручную. Поэтому рекомендуется, чтобы патчи в наборе не могли конфликтовать друг с другом, в противном случае включения/выключения патчей станут большой проблемой. Также следует быть осторожным при удалении патчей, т.к. код может быть потерян навсегда.

Поскольку MQ это расширение, вы должны явно разрешить его в конфигурации Mercurial. Для того чтобы включить MQ, отредактируйте файл `~/.hgrc`, и добавьте строки, представленные на рисунке 47.

```
[extensions]
hgext.mq =
```

Рисунок 47. Добавление расширения MQ для Mercurial

Вы можете использовать MQ с любым Mercurial репозиторием. Чтобы начать работу, подготовьте репозиторий, используя команду `qinit`, как показано на рисунке 48 (её нужно выполнить в корневой папке репозитория).

```
$ hg qinit
```

Рисунок 48. Пример использования команды `qinit`

Эта команда создает пустую папку с именем `.hg/patches`, где MQ будет хранить свои метаданные. Если ваша работа над патчами длительна и серьезна, вы можете захотеть, чтобы состояние очереди патчей версионировалось, служа своим миним-репозиторием. Для этого команде `qinit` нужно передать опцию `-c`.

Работу над новым патчем начинает команда `qnew`, которой надо указать имя патча (см. рис. 49). Также можно использовать опцию `-m` и указать в кавычках сообщение патча (которое станет коммит-сообщением при преобразовании патча в коммит). Команда `qrefresh` позволяет обновить патч, заменив его содержимое используя текущее содержание рабочего каталога – то есть, по сути, отредактировать «коммит». Она работает с последним активным патчем – верхним в «очереди» примененных патчей. На рисунке 49 правки в исходный код программы вносятся при помощи консольного текстового редактора `vi`.

```
$ hg qnew add_pregtextarea
$ vi pregtextarea.php
```

```

$ hg qrefresh
$ hg qnew add_tree_tool
$ vi tree_tool.php
$ hg qrefresh
$ hg qnew add_testing_tool
$ vi testing_tool.php
$ vi tests/testing_tool_test.php
$ hg qrefresh

```



Рисунок 49. Пример создания нескольких патчей

Посмотреть текущее состояние очереди патчей можно с помощью команды `qseries` (показывает все патчи) и `qapplied` (показывает только примененные патчи). Вы можете выключить (т.е. убрать из текущего состояния рабочего каталога, но не удалить) патч используя команду `qpop`, а также включить ранее выключенный патч поверх очереди – командой `qpush`. Обе эти команды имеют опцию `-a` для простого выключения или включения всех патчей (например, можно выключить все патчи, затянуть изменения с апстрима и включить патчи снова поверх них). Работа этих команд влияет состояние рабочего каталога (какие изменения реально есть в коде) и на вывод `qapplied`, но не `qseries`. Посмотрим список всех доступных патчей, которые были ранее созданы (см. рис. 50).

```

$ hg qseries
add_pregtextarea
add_tree_tool
add_testing_tool

```

Рисунок 50. Просмотр списка всех доступных патчей

Стоит отметить, что только что созданные патчи являются сразу же примененными (см. рис. 51).

```

$ hg qapplied
add_pregtextarea
add_tree_tool
add_testing_tool

```

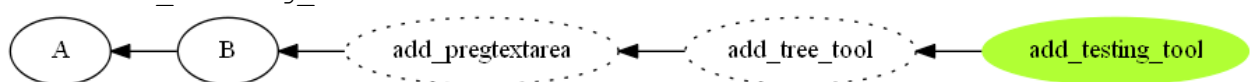


Рисунок 51. Просмотр списка примененных патчей

Допустим, нам нужно отредактировать патч `add_tree_tool`, который находится в нашем примере посередине в списке патчей. Для этого сначала выключим верхний патч, используя команду `hg qpop` (см. рис. 52).

```
$ hg qpop
popping add_testing_tool
now at: add_tree_tool
$ hg qapplied
add_pregtextarea
add_tree_tool
```



Рисунок 52. Выключение верхнего патча

Теперь верхним патчем является `add_tree_tool`, отредактируем его. Чтобы посмотреть разницу между патчем и нашими только что внесенными изменениями, воспользуемся командой `hg qdiff` (см. рис. 53).

```
$ hg qdiff
diff -r ea4378fce999 question/type/preg/edit_preg_form.php
--- a/preg/edit_preg_form.php Mon Feb 06 23:13:10 2017 +0300
+++ b/preg/edit_preg_form.php Mon Feb 06 23:23:37 2017 +0300
@@ -325,9 +325,9 @@
         $data['usecharhint'] = false;
     }

-    if (!array_key_exists('uselexemhint', $data)) {
+    /*if (!array_key_exists('uselexemhint', $data)) {
        $data['uselexemhint'] = false;
-    }
+    }*/

    $i = 0;
    question_bank::load_question_definition_classes($this->qtype());
```

Рисунок 53. Редактирования патча `add_tree_tool`

Обновим патч и посмотрим текущее состояние патчей (см. рис. 54)

```
$ hg qrefresh
$ hg qpush
applying add_testing_tool
now at: add_testing_tool
$ hg qapplied
add_pregtextarea
add_tree_tool
add_testing_tool
```

Рисунок 54. Обновление отредактированного патча

Рассмотрим еще одну ситуацию: нам нужно вытянуть свежие изменения из апстрима. Выключим все патчи, посмотрим текущее состояние патчей. Затянем изменения из апстрима, включим все патчи снова (см. рис. 55).

```
$ hg qpop -a
popping add_testing_tool
popping add_tree_tool
```



```

popping add_pregtextarea
patch queue now empty
$ hg pull
$ hg qpush -a
applying add_pregtextarea
applying add_tree_tool
applying add_testing_tool
now at: add_testing_tool

```

Рисунок 55. Затягивание изменений из апстрима

Преобразовать патч в коммит, сделав его частью постоянной истории можно с помощью команды `qfinish`. Для удаления, оказавшегося ненужным (или принятого апстрим отдельным коммитом) патча есть команда `qdelete`, однако пользоваться ей надо с осторожностью – восстановить патч будет невозможно. Лучше поддержать патч в выключенном состоянии какое-то время, чтобы убедиться, что он действительно больше не нужен.

Из описанного выше примера у нас осталось три патча. Сделаем первый патч коммитом, как показано на рисунке 56.

```

$ hg qpop -a
popping add_testing_tool
popping add_tree_tool
popping add_pregtextarea
patch queue now empty
$ hg push
applying add_pregtextarea
now at: add_pregtextarea
$ hg qfinish tip
$ hg qpush -a
applying add_tree_tool
applying add_testing_tool
now at: add_testing_tool

```



Рисунок 56. Преобразование патча в коммит

Удалим второй патч, он нам больше не нужен (см. рис. 57).

```

$ hg qpop -a
popping add_testing_tool
popping add_tree_tool
patch queue now empty
$ hg qdelete add_tree_tool

```

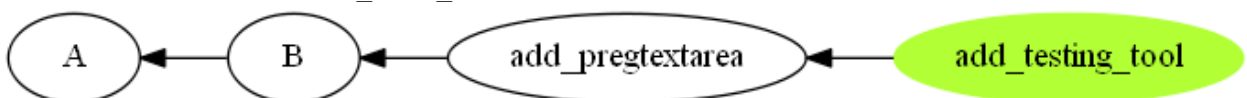


Рисунок 57. Удаление патча

По умолчанию MQ держит патчи локально в специальном каталоге, поэтому при выполнении стандартных команд `push` и `pull` обмен патчами производиться не будет. Используя опцию `-c` при инициализации MQ вы можете создать отдельный репозиторий патчей и выполнять на нем

команды `push/pull`, однако если над очередью патчей работает группа программистов это неудобно. Вместо этого можно заставить MQ использовать возможности секретной фазы коммитов Mercurial для работы с патчами, используя файл конфигурации репозитория, добавив в него строки, представленные на рисунке 58.

```
[mq]
secret = True
```

Рисунок 58. Включение использования секретной фазы коммитов

В этом случае команды `push` и `pull` будут работать с патчами, передавая их между репозиториями (которым следует быть не публикуемыми), сохраняя изменяемость патчей благодаря существованию в секретной фазе коммита.

При управлении большой очередью патчей могут возникнуть вопросы группировки их, для совместного применения или отключения. Для этого можно использовать так называемых «охранников», которые могут быть назначены каждому патчу. Охранники для патча могут быть позитивными (включают патч если охранник включен) и негативными (выключают патч если охранник включен), при этом один охранник для разных патчей может являться позитивным и негативным. Если для патча срабатывают и позитивный и негативный охранники, то негативный выигрывает, т.е. патч не включается.

Назначить охранников патчу можно командой `qguard`, при этом для позитивного охранника перед именем ставится знак плюс, а для негативного – минус. Эта команда всегда переписывает текущих охранников патча, затирая все предыдущие назначения. Без параметров `qguard` может показать список охранников текущего (верхнего) патча. Набор действующих сейчас охранников можно задать командой `qselect`, при этом имена охранников перечисляются без указания знаков плюс и минус перед ними.

Назначим для патчей из примера, описанного выше, позитивных и негативных охранников. Для начала убедимся, что охранников у патча еще нет (см. рис. 59).

```
$ hg qapplied
add_pregtextarea
add_tree_tool
add_testing_tool
$ hg qguard
add_testing_tool: unguarded
$ hg qselect
```

```
no active guards
```

Рисунок 59. Проверка охранников для патчей

Назначим позитивного охранника первому патчу, как показано на рисунке 60. Чтобы установить позитивного охранника на патч, нужно использовать префикс «+» перед именем охранника.

```
$ hg qguard +guard1
$ hg qguard
add_testing_tool: +guard1
```

Рисунок 60. Назначение позитивного охранника патчу add_testing_tool

Назначим негативного охранника второму патчу, как показано на рисунке 61. Чтобы установить негативного охранника на патч, нужно использовать префикс «-» перед именем охранника. Обратите внимание, использование префикса «--» перед аргументами команды `hg qguard` необходимо, чтобы Mercurial не интерпретировал текст `-negative_guard` как одну из возможных опций.

```
$ hg qguard -- add_tree_tool -guard2
```

Рисунок 61. Назначение негативного охранника патчу add_tree_tool

Назначим и позитивного и негативного охранника третьему патчу, как показано на рисунке 62. Стоит отметить, что команда `hg qguard` устанавливает охранников на патч, при этом удаляя предыдущих охранников. Каждый раз необходимо давать полный перечень охранников для этого патча. Это значит, что при установке на патч охранников командой `hg qguard +first +second`, а затем `hg qguard +third` для того же патча, единственным установленным охранником будет `+third`.

```
$ hg qguard -- add_pregtextarea -guard1 +guard2
```

Рисунок 62. Назначение позитивного и негативного охранников патчу add_preg_textarea

Список охранников Mercurial сохраняет в файле `series`. Формат записи охранников в этом файле позволяет без особого труда читать и править их руками через текстовые редакторы. Посмотрим список получившихся охранников (см. рис. 63).

```
$ cat .hg/patches/series
add_preg_textarea #-guard1 #+guard2
add_tree_tool #-guard2
add_testing_tool #+guard1
```

Рисунок 63. Список охранников в файле series

Допустим, необходимо применить все патчи, для которых назначен охранник `guard1`. Для начала необходимо определить, что `guard1` является

действующим охранником, передав его в качестве параметра команде `qselect`. Команда `qpush` позволит применить нужные нам патчи: для которых `guard1` назначен как позитивный охранник (патч `add_testing_tool`) или не назначен вовсе (патч `add_tree_tool`) (см. рис. 64).

```
$ hg qselect guard1
number of guarded, applied patches has changed from 2 to 1
$ hg qselect
guard1
$ hg qpop -a
popping add_testing_tool
popping add_tree_tool
popping add_pregtextarea
patch queue now empty
$ hg qpush -a
applying add_tree_tool
applying add_testing_tool
now at: add_testing_tool
```

Рисунок 64. Применение патчей для охранника `guard1`

Возможно применить патчи сразу для нескольких охранников. Укажем в качестве действующих охранников `guard1` и `guard2` и применим соответствующие патчи (см. рис. 65).

```
$ hg qselect guard1 guard2
number of unguarded, unapplied patches has changed from 2 to 1
$ hg qselect
guard1
guard2
$ hg qpop -a
popping add_testing_tool
popping add_tree_tool
popping add_pregtextarea
patch queue now empty
$ hg qpush -a
applying add_testing_tool
now at: add_testing_tool
```

Рисунок 65. Применение патчей для охранника `guard1` и `guard2`

Из рисунка 65 видно, что негативные охранники имеют приоритет над позитивными. В итоге был применен только патч `add_testing_tool`, имеющий одного позитивного охранника, остальные патчи имели негативных охранников и применены не были.

Учебное издание

Олег Александрович **Сычев**
Григорий Владимирович **Терехов**

**МАНИПУЛИРОВАНИЕ ИЗМЕНЕНИЯМИ ПРОГРАММНОГО КОДА И ИХ
ОТМЕНА**

Методические указания

Темплан ____ г. (учебники и учебные пособия). Поз. № ____.
Подписано в печать 00.00.2020. Формат 60х84 1/16. Бумага газетная.
Гарнитура Times. Печать офсетная. Усл. печ. л. 4. Уч.-изд. л. 5,05.
Тираж ____ экз. Заказ

Волгоградский государственный технический университет.
400005, г. Волгоград, просп. В. И. Ленина, 28, корп. 1.

Отпечатано в типографии ИУНЛ ВолгГТУ.
400005, г. Волгоград, просп. В. И. Ленина, 28, корп. 7.